Dodona 01 — cluster sanity

go / no-go, no load 2026-10-10 16:56 UTC

Objective

Does the cluster serve a real completion, and does it go THROUGH the control plane rather than past it?

Falsified by: orch_requests_total not moving, or a pool not answering

The promises

SLO	target	direction
TTFT p95	1.5s	max
turn p95	30s	max
preemptions	0	max
shed rate at target load	0.05	max
concurrent sessions	derived in 02	min

From benchmarking/slos.py. Session target is derived in 02.

Concurrency target not yet measured — run 02-capacity.

1. Endpoints

PASS gateway /metrics

PASS gateway /v1/models

PASS prefill /metrics

PASS decode /metrics

2. Model and pools

PASS gateway advertises: Qwen/Qwen2.5-7B-Instruct-AWQ

pool	running	waiting	prompt tok	gen tok
prefill	0	0	176,647	66
decode	0	0	176,647	132

prompt tok 0 on a pool means nothing was ever routed to it.

PASS prefill generated 66 tokens (expected near 0, max_tokens=1)

3. Capacity

quantity	value	where it comes from
KV per decode pod	4.30 GiB	pod log: Available KV cache memory
full-length seqs / pod	4.9	pod log: Maximum concurrency
decode replicas	1	kubectl get deploy vllm-decode
decode pool concurrency	4.9	seqs/pod x replicas

A pool this small is the point: a policy needs something to be scarce.

PASS pool of 4.9 is small enough for KV to bind

4. Does a request traverse the gateway

PASS orch_requests_total moved 66 -> 67

PASS completion returned in 0.326s

field	value
content	'ok'

via=local means our cluster served it; via=overflow means it was bursted out.

5. The KV hop — did blocks move?

```
PASS  the router RECORDED a hop
```

field	value
src	prefill-0
dst	decode-0
req_id	http
tokens	16
backend	mooncake
prefix	802848469d1623df

A record, not evidence of bytes. vLLM counts the bytes itself.

counter	value	means
external_prefix_cache_queries	52,523	times the connector was asked
external_prefix_cache_hits	0	times it returned blocks
prompt tokens from transfer	0	tokens that arrived as KV instead of being computed
LOCAL prefix cache hits	124,160	the pod's own cache — not the connector

queries > 0 with hits == 0 means the connector is wired up, is being consulted, and is moving nothing.

FAIL the KV connector has delivered at least one block

The connector was consulted 52,523 times and returned
nothing. Decode recomputes every prompt, so the hop is a record.

5b. Does decode re-prefill?

pool	prompt tok	prefix hits	queries	hit rate
prefill	176,683	124,160	176,683	70.3%
decode	176,683	124,160	176,683	70.3%

Decode loading blocks rather than recomputing shows as a high hit rate.

FAIL decode processes fewer prompt tokens than prefill

Both pools report ~176,683 prompt tokens. queue.py sends the same
prompt to both legs, so this is expected — and it means decode is
NOT skipping prompt work. Confirm with TTFT on a long prompt.

6. Was anything scarce

pool	kv_cache_usage_perc	vs 0.90 threshold
prefill	0.0000	BELOW
decode	0.0000	BELOW

Idle cluster: near zero is expected. make load is what moves this.

An idle reading is not a verdict on the policy — see make load.

7. Where the work lands

7b. The slice budget

component	GiB
slice x util	10.80
weights	5.20
activations + CUDA graphs	1.30
KV cache	4.30

Overhead is budget minus weights minus KV. Measured, not predicted.

8. Latency, short prompt

metric	seconds
n	10
min	0.241
p50	0.305
p95	0.331
max	0.331

Through the tunnel. Shape only — use make load to quote numbers.

9. Latency vs prompt length

prompt tokens	p50 seconds
256	0.318
1,024	0.399
4,096	0.359
8,192	0.446
12,288	0.539

Warmup discarded, so these are cache-warm numbers.

PASS all 5 prompt lengths completed

PASS latency grew 1.7x while the prompt grew 48x (256 -> 12,288 tokens)

10. Grafana

vLLM — KV, running/waiting, TTFT, prefix cache

`http://127.0.0.1:31495/d/dodona-vllm?from=now-30m&to=now`

Gateway — admit, shed, place, latency by stage

`http://127.0.0.1:31495/d/dodona-gateway?from=now-30m&to=now`

Router — pick, sticky, KV hop

`http://127.0.0.1:31495/d/dodona-router?from=now-30m&to=now`

KEDA — scaler value vs replicas

`http://127.0.0.1:31495/d/dodona-keda?from=now-30m&to=now`

HAMi — device memory vs the slice cap

`http://127.0.0.1:31495/d/dodona-hami?from=now-30m&to=now`

explore: KV usage per pod

`http://127.0.0.1:31495/explore?left=%7B%22datasource%22%3A%20%22Prometheus%22%2C%20%22queries%22%3A%20%5B%7B%22`

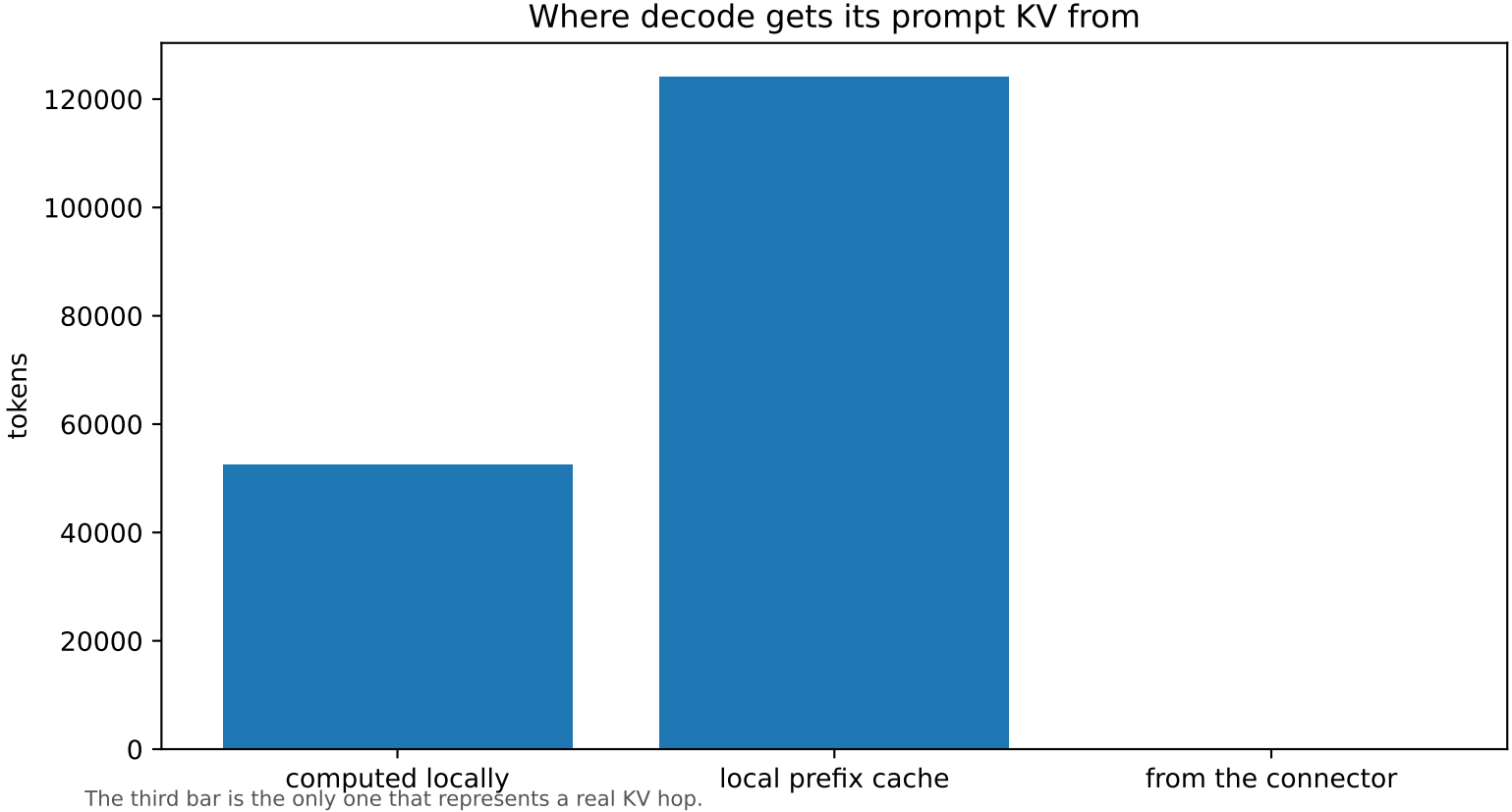
explore: queue depth per pod

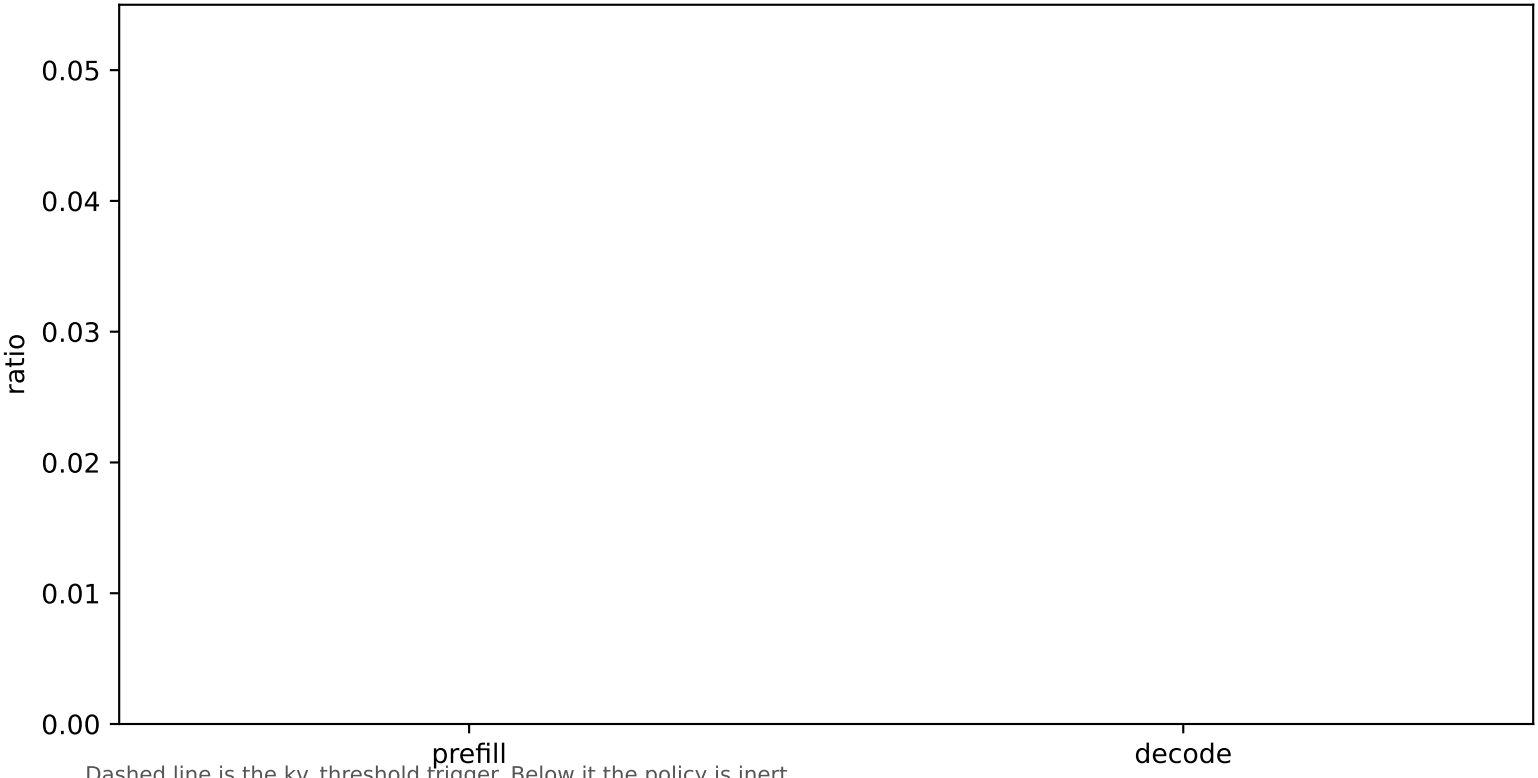
`http://127.0.0.1:31495/explore?left=%7B%22datasource%22%3A%20%22Prometheus%22%2C%20%22queries%22%3A%20%5B%7B%22`

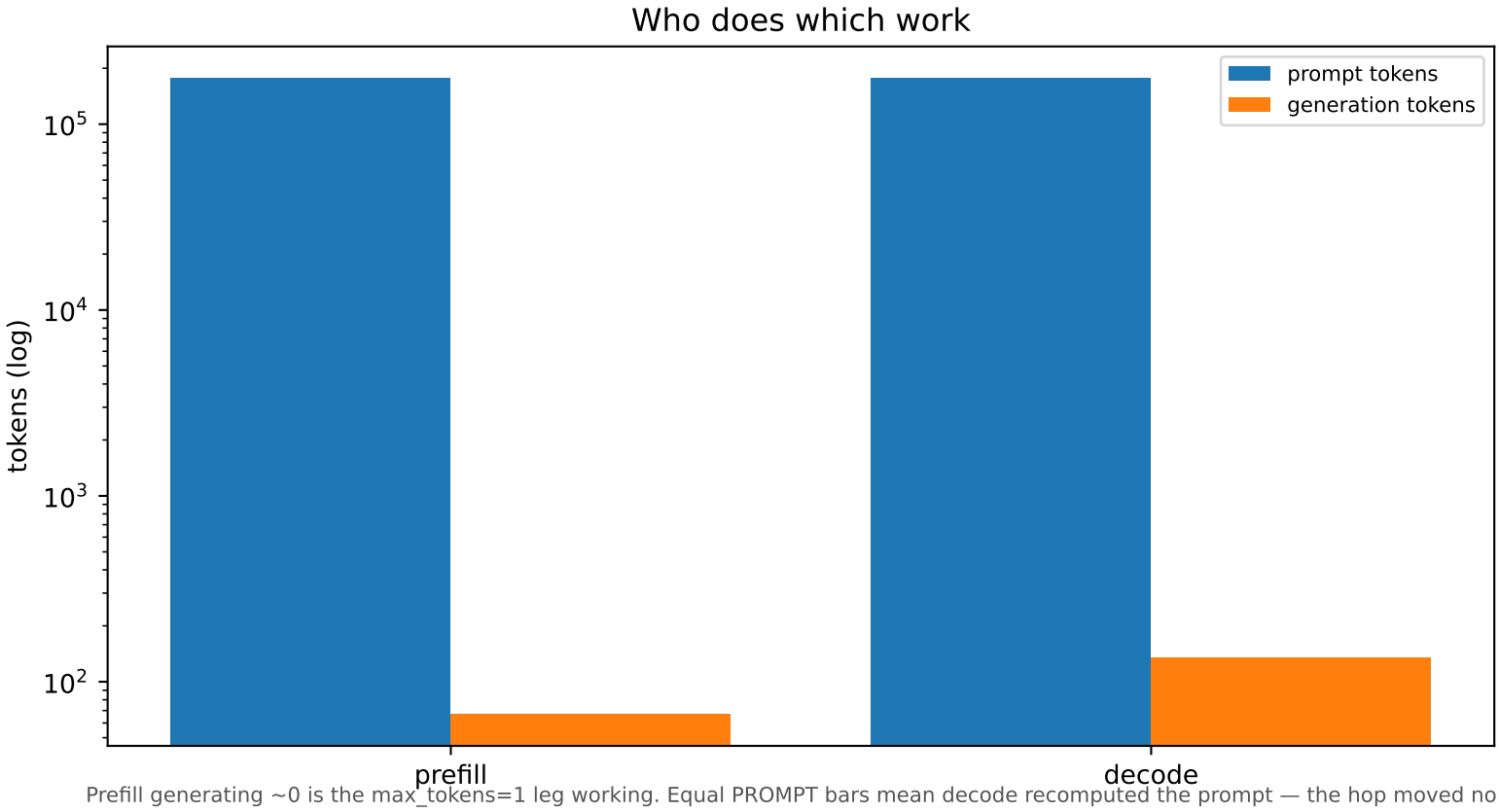
explore: gateway shed by reason

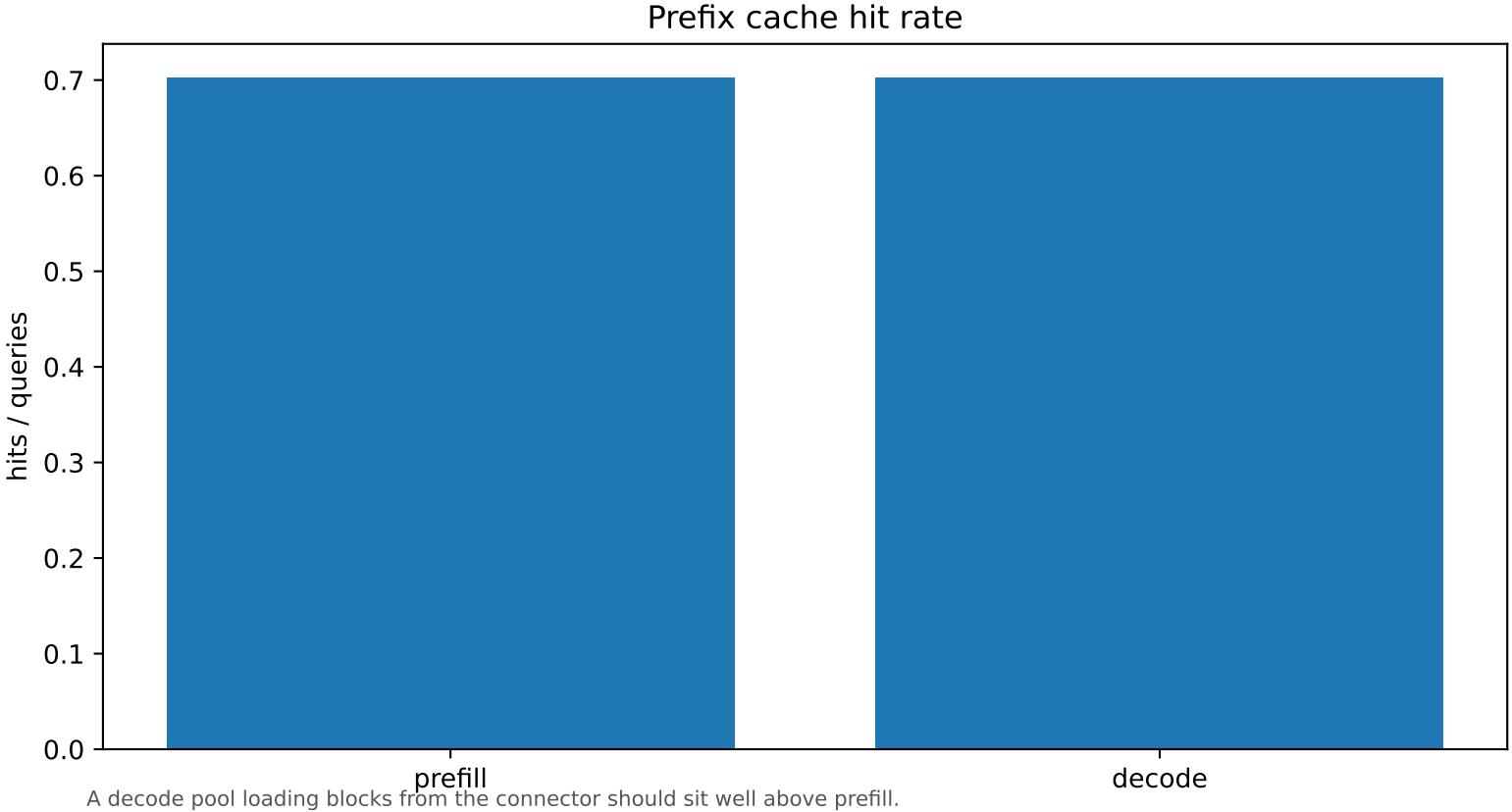
```
http://127.0.0.1:31495/explore?left=%7B%22datasource%22%3A%20%22Prometheus%22%2C%20%22queries%22%3A%20%5B%7B%22
```

Grafana password is printed by make observability.

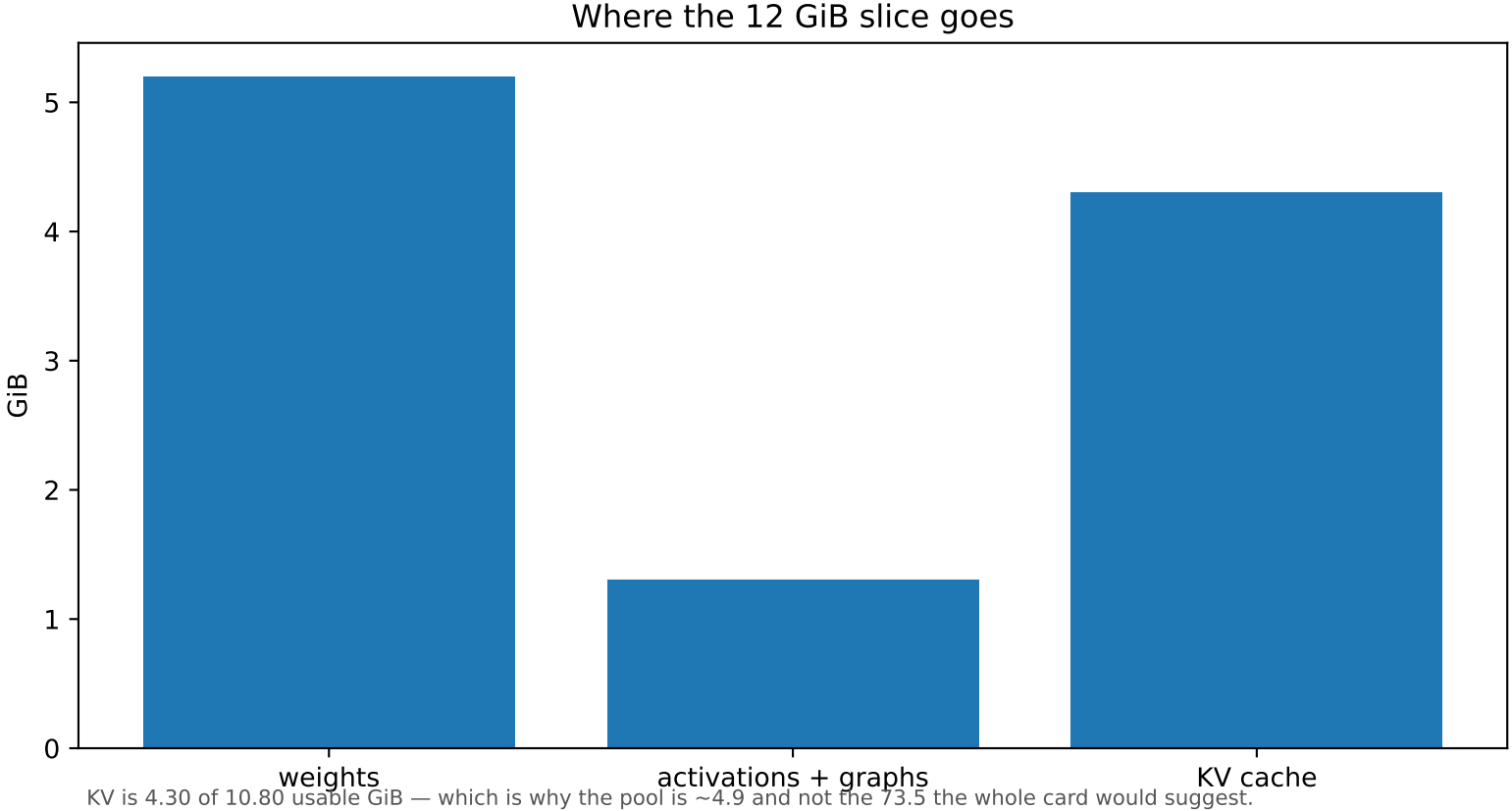

Where decode gets its prompt KV from
tokens
0
20000
40000
60000
80000
100000
120000
computed locally
local prefix cache
from the connector
The third bar is the only one that represents a real KV hop.

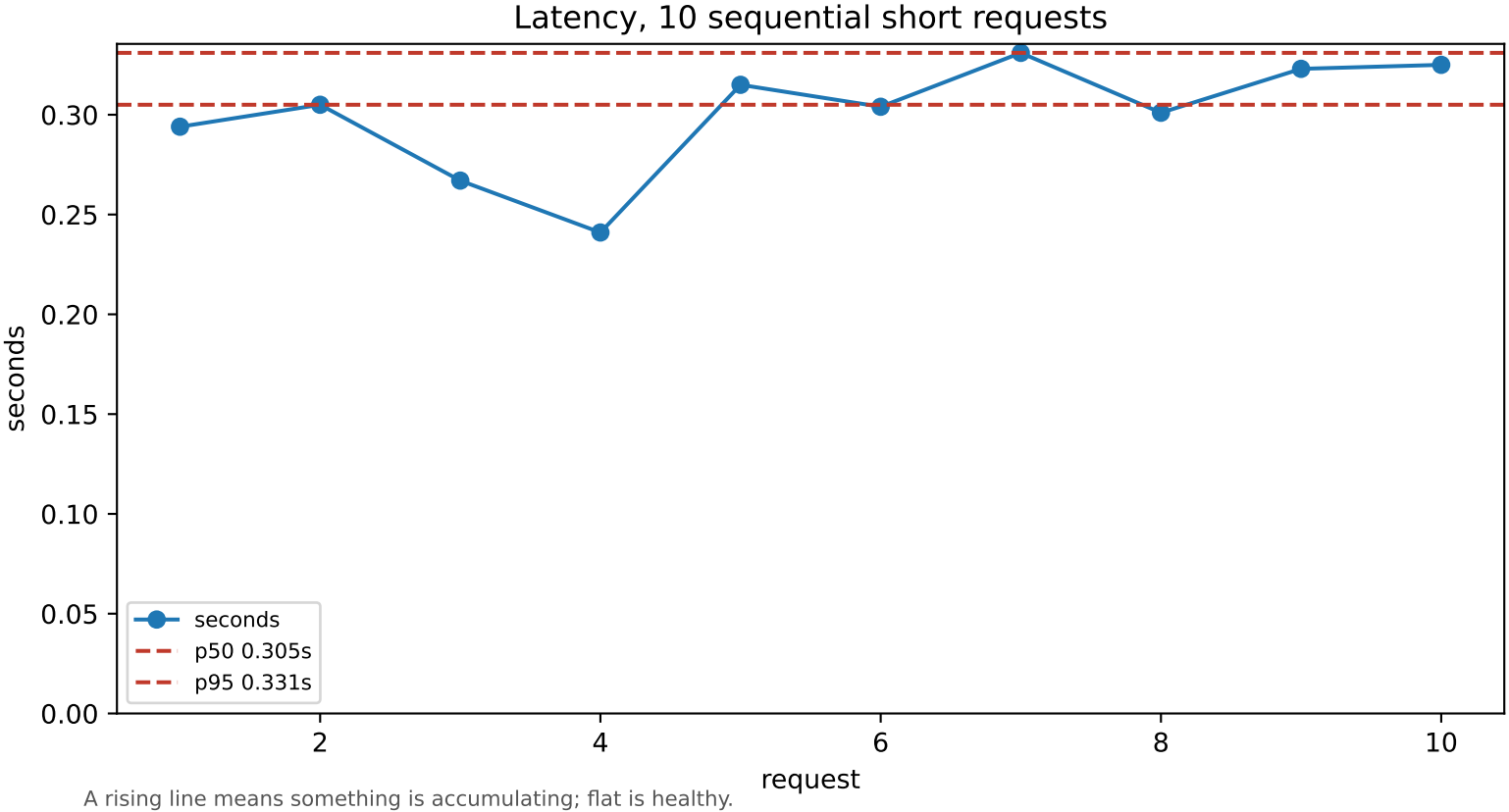

KV usage vs the 0.90 admission threshold
ratio
0.05
0.04
0.03
0.02
0.01
0.00
prefill
decode
Dashed line is the kv_threshold trigger. Below it the policy is inert.


Who does which work
prompt tokens
generation tokens
tokens (log)
10^5
10^4
10^3
10^2
prefill
decode
Prefill generating ~0 is the max_tokens=1 leg working. Equal PROMPT bars mean decode recomputed the prompt — the hop moved no bl

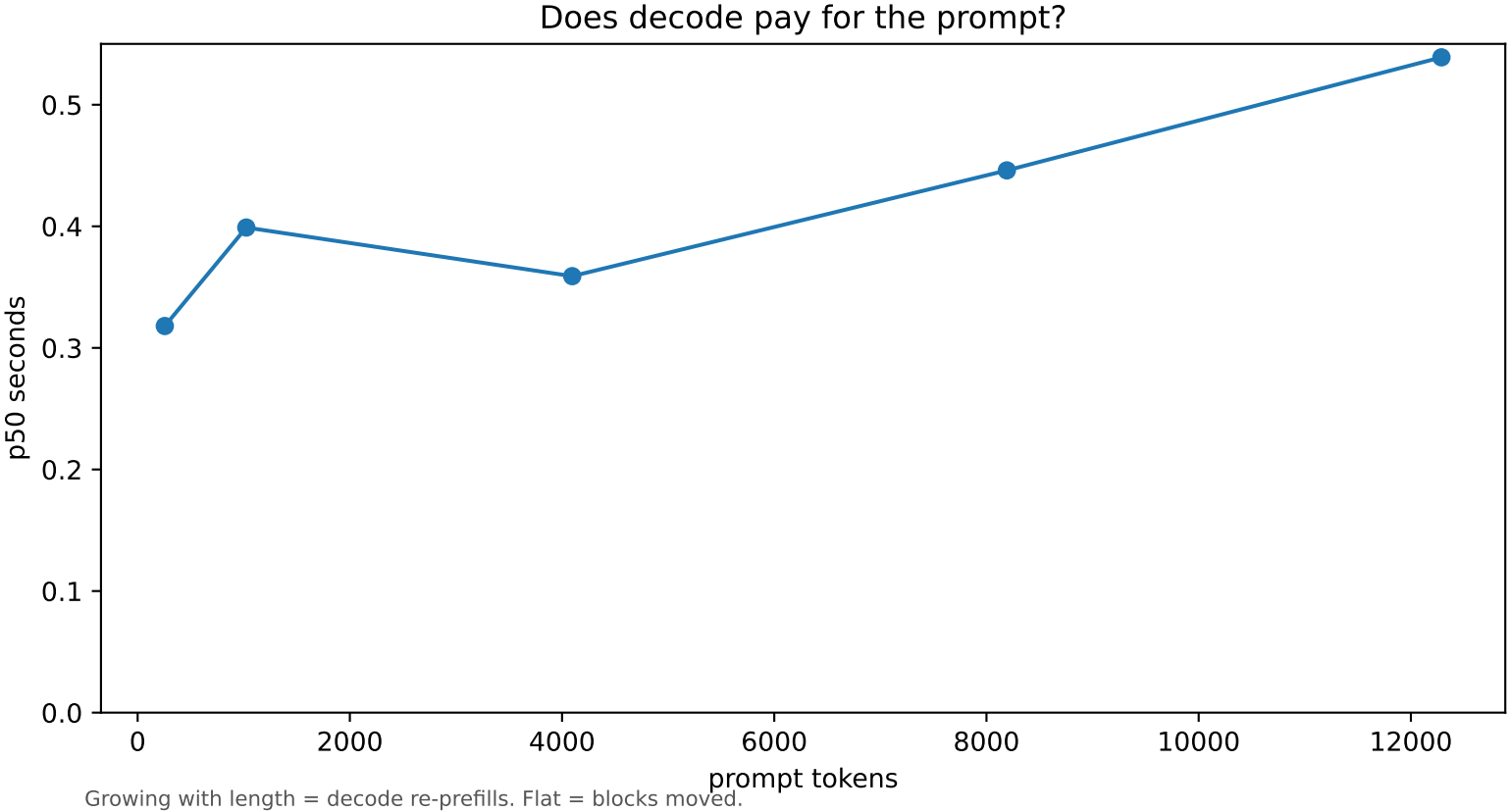


A decode pool loading blocks from the connector should sit well above prefill.


Where the 12 GiB slice goes
GiB
0
1
2
3
4
5
weights
activations + graphs
KV cache
KV is 4.30 of 10.80 usable GiB — which is why the pool is ~4.9 and not the 73.5 the whole card would suggest.



A rising line means something is accumulating; flat is healthy.


Does decode pay for the prompt?
p50 seconds
0.0
0.1
0.2
0.3
0.4
0.5
0
2000
4000
6000
8000
10000
12000
prompt tokens
Growing with length = decode re-prefills. Flat = blocks moved.